



Universidad de Cuenca

Facultad de Ingeniería
Carrera de Ingeniería en Telecomunicaciones

Prácticas Laborales

Henry Maldonado - henrry.maldonado@ucuenca.edu.ec

Informe de Prácticas Laborales: Descripción de la librería para el uso de tarjetas micro SD

Resumen

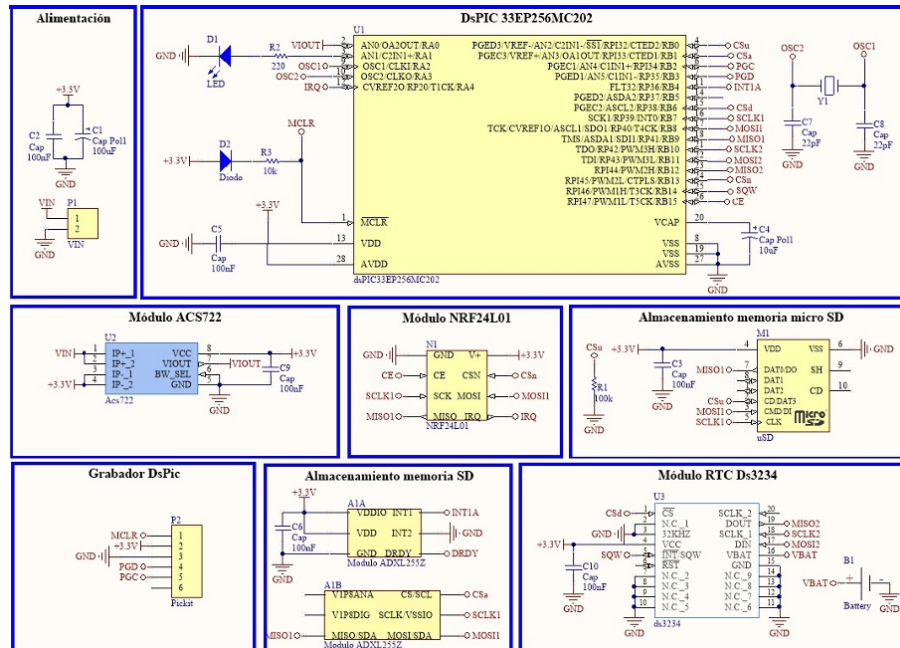
En el presente informe se detalla el funcionamiento de la librería para el uso de tarjetas micro SD usando el DSPIC33EP256MC202. Para lo cual es necesario inicializar las tarjetas para que usen el protocolo de comunicación SPI. Además, se explica la implementación de un programa ejemplo para testear el funcionamiento tanto de tarjetas micro SD Kingston como SanDisk (lectura y escritura).

Los resultados obtenidos de estas pruebas muestran un correcto funcionamiento de la librería con las memorias Kingston. Sin embargo, se identificó un inconveniente al intentar leer datos de tarjetas SanDisk, a pesar de que la inicialización y escritura se llevan a cabo sin errores aparentes.

1. Hardware

El hardware usado para el testeo de la librería de las tarjetas microSD es el que se muestra en la figura [Figura 1](#).

Figura 1. Diagrama esquemático de un nodo sensor.



De la figura [Figura 1](#) nos interesa la comunicación SPI entre el DSPIC y el zócalo para las tarjetas SD. La configuración para SPI 1 según el datasheet del DSPIC es la siguiente:

- **PIN RB8:** Se utiliza como MOSI (Master Out Slave In), es decir, para la transmisión de datos desde el maestro (dsPIC) hacia el esclavo (dispositivo SPI).
- **PIN RB9:** Se configura como MISO (Master In Slave Out), utilizado para la recepción de datos desde el esclavo hacia el maestro.
- **PIN RB0:** Se emplea como el control del Chip Select (CS), que indica al esclavo cuándo debe prestar atención a las señales del maestro.
- **PIN RB7:** Se utiliza como señal de reloj (SCK), que proporciona la sincronización temporal para la comunicación entre el maestro y el esclavo.

Los pines específicos para la comunicación SPI para el zócalo de las tarjetas microSD ya viene especificado por el fabricante, es importante señalar que en el esquemático de la figura [Figura 1](#) no se ha conectado el pin SH que es usado para reconocer si una tarjeta microSD se ha insertado en el zócalo.

2. Software

La librería para el uso de las tarjetas microSD está compuesta por la librería para la comunicación SPI y la librería para la inicialización y funcionamiento de la memoria SD.

2.1. Comunicación SPI

- ✓ Librería `spiSD`: es la encargada de establecer las configuraciones de la comunicación SPI del DSPIC en la función `SPISD_Init()`, en [Librería SPI](#) se presenta la funcionalidad del comando para la configuración SPI, para modificar frecuencias de operación, establecer modo maestro o esclavo, realizar lectura y escritura mediante SPI.

Además, la librería `spiSD` implementa la función para enviar datos mediante SPI, esto se realiza en la función `SPISD_Write()`.

2.2. Inicialización y funcionamiento de la memoria SD (Archivos `sdcard`)

- ✓ Librería `sdcard`: Esta librería es la encargada de realizar el proceso de inicialización de la memoria para establecer el modo de comunicación SPI, ya que, por defecto, la memoria SD estará en modo SD (Secure Digital)

El proceso para la inicialización es el siguiente:

1. Se intenta inicializar la memoria SD usando la función `SD_Init_Try()`, a la cual se le envía como parámetro la cantidad de veces que se intentará inicializar la memoria SD, basta con que se inicialice una sola vez correctamente la tarjeta SD entonces la función retornará `SUCCESSFUL_INIT`.
2. La función `SD_Init_Try()` nombrada anteriormente, llama a la función `SD_Init()`, misma que contiene todo el proceso de inicialización de la memoria SD, el cual es el siguiente:
 - Se coloca como salida al CHIP SELECT, ya que el dsPIC será el master en la comunicación SPI y enviará comandos hacia la memoria microSD.

- Se llama a la función `Release_SD()` para establecer en 1 el CHIP SELECT, puesto que por el momento no se va a leer ni escribir en la memoria microSD.
- Se llama a la función `SPISD_Init(SLOW)` para definir una frecuencia operación baja, en [1] y [2] se menciona que se debe colocar una frecuencia de entre 100 a 400 KHz para la inicialización de la memoria microSD, sin embargo, en el programa realizado la frecuencia mínima es de 625 KHz. Así como también que es necesario dar un mínimo de 74 ciclos de reloj previos a la inicialización, para ello, en el programa se crea un for en el cual se aseguran 80 ciclos de reloj.
- Ahora, se empieza con el proceso de envío de comandos hacia la memoria SD mediante SPI para establecer la comunicación SPI en la micro SD e inicializarla, para ello es necesario conocer qué son los comandos (Sección 2.3) y se sigue el esquema que se muestra en la figura Figura 2.
 - a) Comando **CMD0**. Es el primer comando a enviar, y se utiliza para colocar la tarjeta SD en un estado inactivo. Este comando es crucial al iniciar la comunicación con la tarjeta SD a través del protocolo SPI.

Tabla 1. Formato de comando CMD0 y respuesta

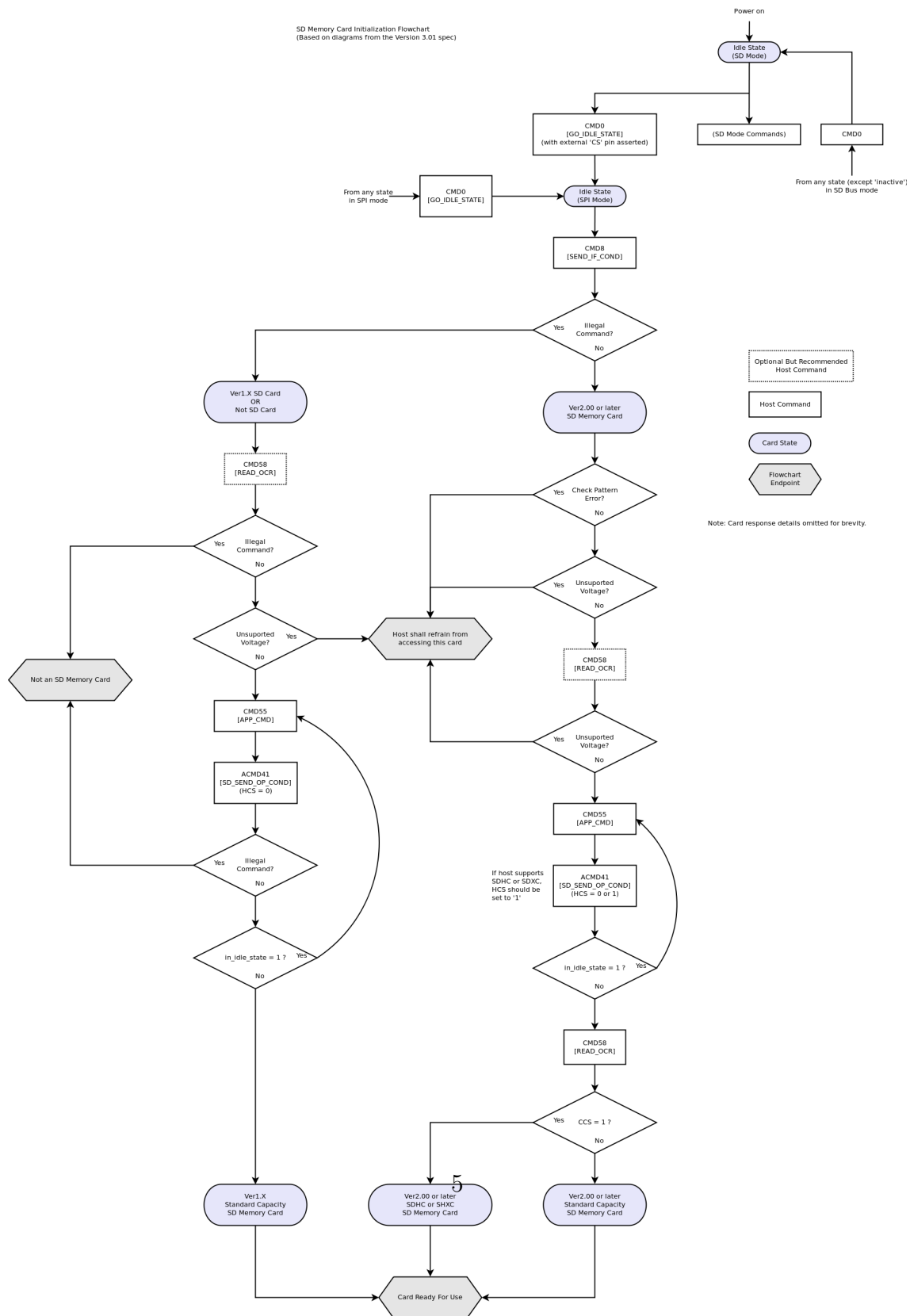
Comando	Argumento	CRC	Respuesta
0x00	0x00000000	0x4A	R1

La respuesta recibida del comando **CMD0** es R1, la cual debe retornar en In Idle State, es decir, solamente con el bit de IDLE STATE activado, caso contrario indicaría que la tarjeta SD no está insertada o que existe algún error.

Adicionalmente, se llama a la función `SD_Ready()` para determinar si la tarjeta está recibiendo la información y retornando respuestas.

- b) Comando **CMD8**: Se envía el comando CMD8 y se verifica primero si la respuesta parcial R1, si R1 está en estado IDLE significa que se trata de una memoria versión 2.0 o superior, caso contrario es posible que la memoria sea antigua.
En cada uno de estos casos el proceso de inicialización cambia.

Figura 2. Esquema de inicialización de tarjeta SD [3].



Ya que el propósito de mejorar la librería es actualizarla para memorias actuales, se analiza el proceso para memorias versión 2.0 o superior.

Es así que, se analiza la respuesta faltante de 32 bits que retorna al enviar el comando CMD8, y se verifica si la memoria es compatible con los rangos de voltaje.

Si el voltaje es compatible, se sigue con el proceso de inicialización.

- c) Comando **CMD58**: Se envía el comando CMD58 y se analiza la respuesta parcial R1 de R3, esta debe estar en estado IDLE si es que no existe ningún error, adicionalmente, se analiza los 32 bits sobrantes de R3 que corresponden al OCR (Registro de condiciones de operación), mismo que contiene los rangos específicos de voltaje con los que opera la memoria micro SD para verificar que sea compatible.
- d) Comando **CMD59**: Se activa la verificación con CRC mediante el comando CMD59, esto quiere decir que es necesario pasar un CRC7 correcto a los comandos que se enviarán posteriormente. Luego, se verifica que la respuesta a este comando, R1, se encuentre en estado IDLE.
- e) Comando **CMD55**: Mediante este comando se indica a la memoria que el siguiente comando que recibirá será un comando específico de aplicación (ACMD).
- f) Comando **ACMD41**: Mediante este comando se indica a la memoria que se va a pasar a modo escritura y lectura, por ello, se analiza la respuesta recibida de la memoria micro SD, la cual debe de estar en estado DONE, lo que significa que todos los bits de la respuesta R1 tienen que estar en cero, es decir, la memoria está lista para usarse.
A partir de este punto, la respuesta R1 deberá retornar siempre con cero, esto indica que la tarjeta ha completado su inicialización y está lista para ser usada, en caso de no retornar con cer significa que ha existido algún error en el comando enviado.
- g) Mediante el comando CMD59 se deshabilita la verificación del CRC y se verifica que la respuesta R1 obtenida de la memoria micro SD se encuentre en cero, esto indica que la microSD está lista.
- h) Comando **CMD16**: Se aplica el comando para setear el tamaño

de bloque de la tarjeta a 512 bytes, en las tarjetas V2 esta ya es una configuración por defecto.

- i) Comando **CMD58**: Se lee la capacidad de la memoria SD el cual es el bit CCS de OCR.
- j) Finalmente, se activa el CHIP SELECT y se aumenta la frecuencia de operación de la comunicación SPI, retornando así **SUCCESSFUL_INIT** de la función **SD_Init()**.

3. Una vez inicializada la tarjeta SD ya se puede realizar lectura y escritura de los sectores. Para ello se definen las siguientes funciones correspondientes a la lectura y escritura respectivamente:

- **SD_Read_Block()**: Esta función lee un sector completo de la tarjeta SD, recibe como parámetro el buffer en donde se desea almacenar la información leída (de 512 bytes) y el sector al cual se va a leer. Dentro de esta función primero se coloca el CS en 0 (función **Select_SD()**), ya que se va a realizar la lectura de la memoria SD, se valida si la memoria es versión 1 o 2 según el CCS, en caso de ser V1 se multiplica el número del sector por 512 para localizar físicamente al sector al cual se va a leer, caso contrario se usa directamente el parámetro recibido para localizar al sector físico de la tarjeta SD. Posteriormente, se envía el comando de lectura (CMD17) con el número de sector a leer y el correspondiente CRC7. La respuesta R1 recibida en el microcontrolador debe indicar el estado DONE, caso contrario existe un error en la lectura.
Esta función llama a la función **SD_Read()** que lee el sector definido, para lo cual se le envía como parámetro el buffer que almacenará la información leída y la cantidad de bytes a leer. Debido a que ya se envió el comando CMD17 es necesario leer el TOKEN que retorna, si este token es correcto se realiza la lectura de los datos y se los almacena en el buffer.
Finalmente, con la función **Release_SD()** se coloca el CS en 1, ya que se ha terminado la lectura de datos.
- **SD_Write_Block()**: Es responsable de escribir un bloque de datos en un sector específico de una tarjeta SD. Para ello requiere del buffer del cual se va a copiar su contenido en la memoria SD y la dirección del sector en el cual se va a copiar esa información. Se coloca el CS en 0 (función **Select_SD()**), ya que se va a realizar escritura sobre la memoria SD. Se ajusta la dirección del sector si es necesario, se

verifica la disponibilidad de la tarjeta SD para escritura, se envía el comando de escritura y el bloque de datos, y procesa las respuestas. Luego, se libera la tarjeta SD (función `Release_SD()`) y devuelve un código de estado indicando si la operación fue exitosa, si hubo un error de CRC, un error de escritura, o un error no especificado.

2.3. Comandos

Los comandos CMD (Command) son instrucciones enviadas desde el controlador host (por ejemplo, un microcontrolador) a una tarjeta SD para realizar diversas operaciones, como inicialización, lectura, escritura, borrado, etc. Estos comandos son parte del protocolo de comunicación establecido por la especificación de la tarjeta SD, en este caso, comunicación SPI.

Para enviar comandos hacia la tarjeta SD se requiere de tres parámetros:

1. Comando
2. Argumento
3. Código de redundancia Cíclica CRC7

Estos son enviados en una estructura de 6 bytes como se muestran en la [Tabla 2](#), como resultado se obtienen respuestas por parte de la tarjeta SD (Sección 2.4).

Tabla 2. Formato de comandos. Extraída de SD specification Part 1 Physical Layer Simplified Specification en la sección 7.3.1.1, descargado de [SD specification Part 1 Physical Layer Simplified Specification](#)

Bit position	47	46	[45: 40]	[39: 8]	[7: 1]	0
Width (bits)	1	1	6	32	7	1
Value	0	1	x	x	x	1
Description	start bit	transmission bit	command index	argument	CRC7	end bit

La transmisión del comando siempre comienza con el bit más a la izquierda de la cadena de bits correspondiente a la palabra de código de comando. De esta forma el proceso para enviar un comando es:

1. Enviar el primer byte, el cual está compuesto por el **start bit**, seguido del **transmission bit** y por el comando (con longitud de 6 bits).
2. Enviar los siguientes 4 bytes de forma individual, mismos que corresponden al argumento.
3. Enviar el último byte, compuesto por el CRC7 (con longitud de 7 bits) seguido del **end bit**.

Una vez definida la estructura de los comandos, se presenta a continuación una lista de los comandos usados en el programa realizado para la comunicación SPI entre un microcontrolador y una tarjeta SD, extraído de [\[3\]](#):

Tabla 3. Comandos usados en el programa

Comando [6 bits]				Argumento [32 bits]	Código CRC [7 bits]	Respuesta
Índice	Patrón de bits	Mnemónico	Descripción del comando			
CMD0	0x00	GO_IDLE_STATE	Reinicia la Tarjeta de Memoria SD	[31:0] Bits de Relleno	0x4A	R1
CMD1	0x01	SEND_OP_COND	Envía información de soporte de capacidad del host y activa el proceso de inicialización de la tarjeta.	[31] Reservado [30] HCS [29:0] Reservado	0x7C	R1
CMD8	0x08	SEND_IF_COND	Envía la condición de la interfaz de la tarjeta de memoria SD que incluye información del voltaje de suministro del host (VHS) y pregunta a la tarjeta a la que se accede si la tarjeta puede funcionar en el rango de voltaje suministrado. Los bits reservados se pondrán a '0'. NOTA: Analizar en la referencia los valores que se requieren para enviar correctamente la VHS	[31:12] Bits reservados [11:8] Tensión de alimentación (VHS) [7:0] Comprobar patrón	0x43	R7
CMD9	0x09	SEND_CSD	Solicita a la tarjeta seleccionada que envíe sus datos específicos de la tarjeta (CSD)	[31:0] Bits de Relleno		R1
CMD10	0x0A	SEND_CID	Solicita a la tarjeta seleccionada que envíe su identificación de tarjeta (CID)	[31:0] Bits de Relleno		R1
CMD13	0x0D	SEND_STATUS	Solicita a la tarjeta seleccionada que envíe su registro de estado.	[31:0] Bits de Relleno		R2
CMD16	0x10	SET_BLOCKLEN	Establece una longitud de bloque (en bytes) para todos los siguientes comandos de bloque (lectura y escritura) de una tarjeta de capacidad estándar. La longitud del bloque de los comandos de lectura y escritura se fija en 512 bytes en las tarjetas SDHC y SDXC.El tamaño de bloque está definido por el comando LOCK_UNLOCK	[31:0] Longitud de Bloque	0x0A	R1
CMD17	0x11	READ_SINGLE_BLOCK	Lee un bloque del tamaño seleccionado por el comando SET_BLOCKLEN	[31:0] Dirección de Datos		R1
CMD24	0x18	WRITE_BLOCK	Escribe un bloque del tamaño seleccionado por el comando SET_BLOCKLEN.	[31:0] Dirección de Datos		R1
CMD55	0x37	APP_CMD	Define en la tarjeta que el siguiente comando es un comando específico de la aplicación en lugar de un comando estándar.	[31:0] Bits de Relleno	0x32	R1
CMD58	0x3A	READ_OCR	Lee el registro OCR de una tarjeta. El bit CCS está asignado a OCR[30].	[31:0] Bits de Relleno	0x7E	R3
CMD59	0x3B	CRC_ON_OFF	Activa o desactiva la opción CRC. Un 1 activa la opción, un 0 la desactiva.	[31:1] Bits de relleno [0] opción CRC	0x41	R1
ACMD13	0x0D	SD_STATUS	Enviar el registro de estado de las tarjetas. La especificación dice que la tarjeta (en modo SPI) responderá con un token R2	[31:0] Bits de Relleno		R2
ACMD41	0x29	SD_SEND_OP_COND	Envía información de soporte de capacidad del host y activa el proceso de inicialización de la tarjeta. Los bits reservados se pondrán a 0.	[31] Bit reservado [30] HCS [29:0] Bits reservados	0x3B	R1
ACMD42	0x2A	SET_CLR_CARD_DETECT	Se encarga de conectar (1) o desconectar (0) la resistencia pull-up de 50 kOhm en CS (pin 1) de la tarjeta. La resistencia pull-up puede ser utilizada para la detección de la tarjeta.	[31:1] Bits reservados [0] set_cd		R1
ACMD51	0x33	SEND_SCR	Lee el registro de configuración SD (SCR).	[31:0] Bits de Relleno		R1

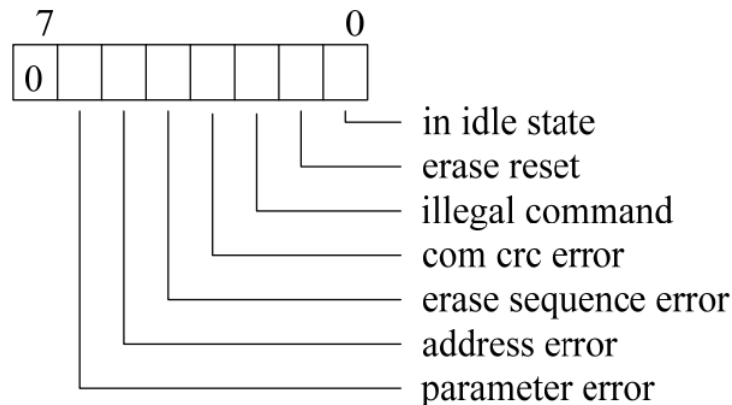
2.4. Respuestas

Las respuestas obtenidas son producto de los comandos enviados hacia la tarjeta SD, al recibir los comandos la memoria retorna al master estas respuestas.

La estructura de estas depende del tipo de comando usado, a continuación se presentan las respuestas más comunes retornadas por las tarjetas SD:

- ✓ Respuesta R1: Esta es la respuesta básica de la tarjeta SD a una instrucción de comando. Es un byte que contiene un estado de operación o un código de error básico.
 - **Bit 0:** En estado de reposo (Idle State): La tarjeta está en estado de reposo y ejecuta el proceso de inicialización.
 - **Bit 1:** Restablecimiento de borrado (Erase Reset): Se ha despejado una secuencia de borrado antes de ejecutarla debido a que se recibió un comando fuera de secuencia de borrado.
 - **Bit 2:** Error de comando ilegal (Illegal Command): Se detectó un código de comando ilegal.
 - **Bit 3:** Error de CRC de comunicación (Communication CRC Error): La verificación de CRC del último comando ha fallado.
 - **Bit 4:** Error de secuencia de borrado (Erase Sequence Error): Se produce un error en la secuencia de comandos de borrado.
 - **Bit 5:** Error de dirección (Address Error): Se utilizó una dirección desalineada que no coincidía con la longitud del bloque en el comando.
 - **Bit 6:** Error de parámetro (Parameter Error): El argumento del comando (por ejemplo, dirección, longitud del bloque) estaba fuera del rango permitido para esta tarjeta.
 - **Bit 7:** MSB (Bit más significativo) - Siempre cero.

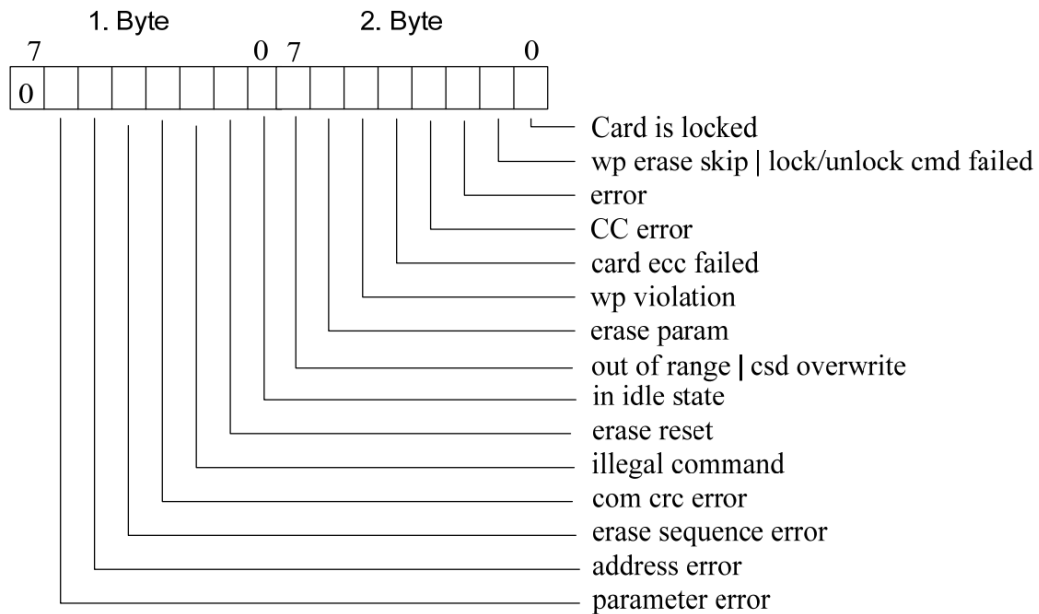
Figura 3. Formato de respuesta R1 [4].



✓ Respuesta R2: Tiene dos bytes de ancho. El primer byte enviado es idéntico a R1. El segundo byte enviado es el siguiente:

- **Bit 0:** Tarjeta bloqueada (Card is locked): Se establece cuando la tarjeta está bloqueada por el usuario. Se reinicia cuando está desbloqueada.
- **Bit 1:** Salto de borrado de protección contra escritura — Fallo en el comando de bloqueo/desbloqueo (Write protect erase skip — lock/unlock command failed): Este bit de estado tiene dos funciones sobrecargadas. Se establece cuando el host intenta borrar un sector protegido contra escritura o comete errores de secuencia o de contraseña durante la operación de bloqueo/desbloqueo de la tarjeta.
- **Bit 2:** Error: Ocurrió un error general o desconocido durante la operación.
- **Bit 3:** Error de CC (CC error): Error interno del controlador de la tarjeta.
- **Bit 4:** Fallo de ECC de la tarjeta (Card ECC failed): Se aplicó ECC interna de la tarjeta pero no se pudo corregir los datos.
- **Bit 5:** Violación de protección contra escritura (Write protect violation): El comando intentó escribir en un bloque protegido contra escritura.
- **Bit 6:** Parámetro de borrado (Erase param): Una selección inválida para el borrado, sectores o grupos.
- **Bit 7:** Fuera de rango — Sobrescritura de CSD (out of range — csd overwrite)

Figura 4. Formato de respuesta R2 [4].

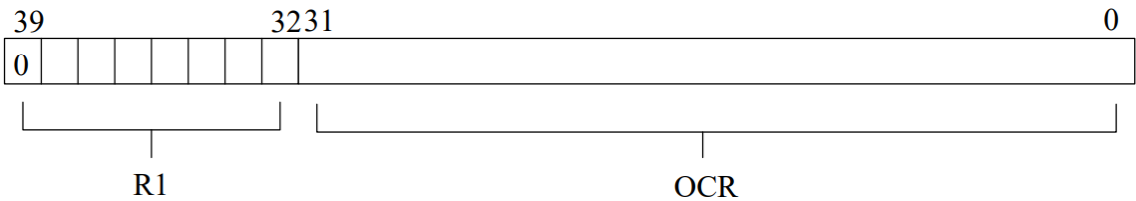


- ✓ Respuesta R3: Cinco bytes de ancho. El primer byte enviado es idéntico a R1. Los cuatro bytes siguientes son los contenidos del registro OCR. Hay algunas cosas importantes a tener en cuenta. Primero, cada bit del 15 al 24 indica un valor de voltaje soportado por esta tarjeta. A continuación, el bit 31 es el estado de encendido de la tarjeta, que es BAJO si la tarjeta no ha terminado la rutina de encendido. Y el bit 30, el estado de capacidad de la tarjeta, que solo es válido si el estado de encendido está establecido, indica que la tarjeta es una tarjeta SD de alta capacidad (SDHC) o una tarjeta SD de capacidad extendida (SCXC), en lugar de una tarjeta SD de capacidad estándar (SDSC).

Tabla 4. Definición de los campos del OCR en la respuesta R3

Posición del bit en el OCR	Definición de los campos del OCR
0-3	Reservado
4	Reservado
5	Reservado
6	Reservado
7	Reservado para Rango de Voltaje Bajo
8	Reservado
9	Reservado
10	Reservado
11	Reservado
12	Reservado
13	Reservado
14	Reservado
15	2.7-2.8
16	2.8-2.9
17	2.9-3.0
18	3.0-3.1
19	3.1-3.2
20	3.2-3.3
21	3.3-3.4
22	3.4-3.5
23	3.5-3.6
24	Cambio a 1.8V Aceptado (S18A)
25-28	Reservado
29	Estado de la tarjeta UHS-II
30	Estado de Capacidad de la Tarjeta (CCS)
31	Bit de estado de encendido de la tarjeta (ocupado)

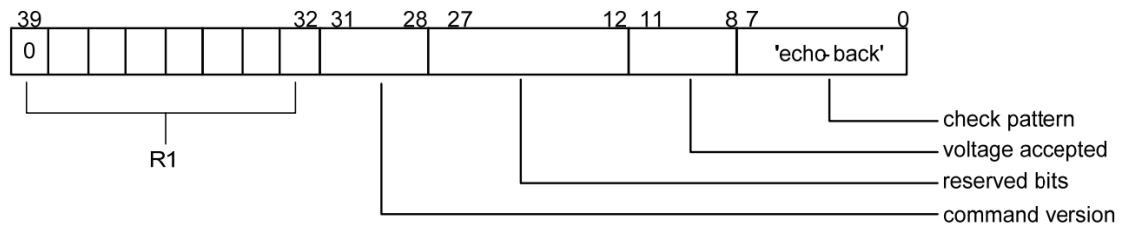
Figura 5. Formato de respuesta R3 [4].



✓ Respuesta R7: Este token de respuesta es enviado por la tarjeta cuando se recibe un comando SEND_IF_COND (CMD8). La longitud de la respuesta es de 5 bytes. La estructura del primer byte (MSB) es idéntica al tipo de respuesta R1. Los otros cuatro bytes contienen información sobre el voltaje de operación de la tarjeta y el eco del patrón de verificación en el argumento, y están especificados por la misma definición que la respuesta R7 en modo SD.

- **Bits 0-7:** Idéntico a R1.
- **Bits 8-39:** Voltaje de operación de la tarjeta y eco del patrón de verificación en el argumento.

Figura 6. Formato de respuesta R7 [4].



3. Código de ejemplo

Una vez analizado la librería se desarrolló un programa para la verificación de inicialización, lectura y escritura de datos de la memoria usando el DSPIC33EP256MC202.

En el programa realizado se ha usado funciones del programa Nodoacelerometro.c para la definición de variables e inicialización de la tarjeta SD, una vez realizada la inicialización se ha creado a función `Ejemplo_uso_SD()` en la cual se realiza la escritura de 512 bytes en un sector completo de la tarjeta SD, posteriormente se realiza la lectura del mismo sector para finalmente escribir la información leída en otro sector, esto permite verificar que tanto la escritura y lectura de la información en el SD es correcta.

Además, se modificó la función `GuardarTramaSD()` con datos de ejemplos para simular la cabecera de la trama, información de tiempo y datos del acelerómetro. Esta función se encarga de escribir una trama completa en la tarjeta SD, cuenta con datos de cabecera, tiempo y datos de aceleración, para lo cual requiere de 2512 bytes o de forma equivalente de 5 sectores. Par verificar el funcionamiento de esta función es necesario analizar la memoria SD con HXD, donde 5 sectores consecutivos deben tener la información enviada.

3.1. Pruebas

Se realizó pruebas de funcionamiento con dos marcas de memorias SD: Kingston y SanDisk.

- ✓ Memorias Kingston: Al realizar pruebas con memorias Kingston de 16 GB y 32 GB se verificó que la inicialización, escritura y lectura de datos es correcta, para ello se ejecutó el programa ejemplo y mediante el software HXD se analizó los sectores utilizados, verificando la información almacenada.
- ✓ Memorias SanDisk: Al realizar pruebas con memorias SanDisk de 16 GB se verificó el proceso de inicialización analizando la respuesta R1 así como también el bit CCS del OCR, ambos resultados fueron exitosos, ya que al finalizar la inicialización de la memoria la respuesta R1 el comando ACMD41 se encontraba en cero, y el CCS estaba en 1 lógico, lo que indica que la memoria utilizada era SDHC.

También se comprobó la escritura usando el programa ejemplo, la cual fue exitosa.

Existió un error en la lectura del sector en este tipo de memoria, luego de revisar el proceso de inicialización se detectó que el error se encuentra en la

respuesta R1 debido al comando CMD17, para que la respuesta sea correcta esta debería estar en 0x00, sin embargo, la respuesta obtenida es 0xFF, lo que indica un error.

Además, a diferencia de las tarjetas Kingston, las tarjetas SanDisk NO reinician el proceso de inicialización al ser reconectadas en el zócalo, para ello es necesario reiniciar la energía en el módulo. Posiblemente, este proceso es propio del fabricante. Esto también puede ser solucionado usando el pin del zócalo de la memoria SD para detectar si la memoria está insertada o no (variable `sdflags.detected`).

Se realizaron varios intentos para solucionar el error de lectura:

- Verificar el proceso de inicialización, es decir, se verificó que la respuesta R1 en el proceso de inicialización se encuentre, ya sea en estado IDLE o DONE (cero).
- Se verificó que el DSPIC reconozca el tipo de memoria usada, para ello se verificó que el pin CCS del OCR sea 1, lo que indica que se trata de una memoria SDHC.
- Se aplicaron varios ciclos de reloj previa y posteriormente al proceso de inicialización, esto mediante bucles for. Esta información fue obtenida en foros; sin embargo, no se tuvo éxito.

Durante la investigación se encontraron posibles soluciones:

- Reducir la frecuencia de la comunicación SPI en el proceso de inicialización a un rango de 100 a 400 KHz, ya que actualmente opera a un mínimo de 625 KHz.
- Verificar los rangos de voltaje asignados para la comunicación SPI, para ello se puede analizar las respuestas R3 debido a los comandos CMD8 y CMD58.

Referencias

- [1] C. Ababei, "Lecture 12: Spi and sd cards," http://www.dejazzer.com/ee379/lecture_notes/lec12_sd_card.pdf, 2013, eE-379 Embedded Systems and Applications, Electrical Engineering Department, University at Buffalo.
- [2] F. Foust. [Online]. Available: http://alumni.cs.ucr.edu/~amitra/sdcard/Additional/sdcard_appnote.foust.pdf

- [3] “Secure digital (sd) card spec and info,” 2024. [Online]. Available: <https://chlazza.nfshost.com/sdcardinfo.html>
- [4] S. S. Part, “Physical layer simplified specification,” *URL: http://academy.cba.mit.edu/classes/networking-communications/SD/SD.pdf (visited on 11/22/2021)*, 1.

4. Anexo:

```
1 void Ejemplo_uso_SD(){
2     /* Funcion que escribe en un sector, luego lee el sector y
3     vuelve a
4     escribir la informacion leida en otro sector, de esta forma se
5     verifica tanto
6     escritura como lectura
7     */
8     unsigned char data_to_write[512]; // datos que se van a
9     escribir
10    unsigned long sector;                // numero del sector a leer o
11    escribir
12    unsigned char valor;                  // byte que se va a escribir
13    en los sectores
14    unsigned char buffer[512];           // variable que almacenara la
15    informacion leida
16
17    // FOR QUE SE ENCARGA DE LLENAR DE VALORES LOS 512 BYTES
18    valor = 1; // Iniciar con el valor 1
19    for (i = 0; i < 512; i++) {
20        data_to_write[i] = valor;
21        valor++; // Incrementa el valor para la pr xima iteraci n
22        // Si el valor alcanza 256, restablecer a 1 para repetir el
23        ciclo
24        if (valor == 256) {
25            valor = 1;
26        }
27    }
28
29    // EJEMPLO DE ESCRITURA EN UN SECTOR DE LA SD
30    sector=2500; // Definicion del sector en donde se va a escribir
31    checkEscSD = SD_Write_Block(data_to_write,sector);
32    if (checkEscSD == DATA_ACCEPTED){
33        LED(1,1);
34    }
35
36    // EJEMPLO DE LECTURA DE UN SECTOR ANTERIORMENTE ESCRITO DE LA
37    SD
38    checkLecSD = SD_Read_Block(buffer, sector);
39    if (checkLecSD==0) {
40        LED(1,1);
41    }
42
43    // SE LEE LOS DATOS PREVIAMENTE ESCRITOS Y SE LOS ESCRIBE
```

```
36     NUEVAMENTE EN OTRO SECTOR
37     // DE ESTA FORMA SE VERIFICA QUE LA LECTURA REA CORRECTA
38     //sector=40001;
39     checkEscSD = SD_Write_Block(buffer,sector+1);
40     if (checkEscSD == DATA_ACCEPTED){
41         LED(1,1);
42     }
43 }
```